

MindPal 推荐模块下一阶段迭代计划

面向版本：v4.4 → v5.0
当前基线：v4.3
核心目标：从“规则可运行的推荐模块”升级为“可评测、可调参、可反馈学习、可解释的个性化推荐系统”。

1. 当前判断

推荐模块 v4.3 已经完成了工程闭环：
用户输入经过情绪分析、风险评估、用户画像加载后，由 RecommendGate 判断是否推荐，再由 ContentRecommender 进行召回、排序、AI Rerank 和推荐理由生成。

因此下一阶段的重点不应是继续简单增加规则，而应该转向四件事：

- 1. 建立推荐评测体系
- 2. 校准推荐门控
- 3. 扩展并结构化内容库
- 4. 引入用户反馈闭环，逐步从规则推荐升级到可学习推荐

推荐模块的下一步核心问题不是“能不能推荐”，而是：

什么时候推荐才合适？推荐什么才有用？用户是否真的接受？系统如何根据反馈变好？

2. 总体路线

下一阶段建议分为 5 个阶段推进：

阶段	版本	目标	优先级
阶段一	v4.4	建立推荐可观测与离线评测体系	最高
阶段二	v4.5	校准 RecommendGate 门控策略	高
阶段三	v4.6	扩展内容库与内容标签体系	高
阶段四	v4.7	建立用户反馈闭环	中高
阶段五	v5.0	引入轻量学习排序模型	中

建议先做 v4.4 和 v4.5。因为没有评测系统之前，继续改推荐算法很容易变成“凭感觉调参”。

3. 阶段一：v4.4 推荐可观测与评测体系

3.1 目标

给推荐模块建立一套“可以复盘、可以打分、可以调参”的评测系统。

当前模块有完整测试，但测试主要验证代码逻辑是否正确。下一步需要补的是推荐质量评测，也就是判断：

- 这一轮是否应该推荐？
- 推荐类型是 hard 还是 soft？
- 推荐内容是否匹配用户情绪？
- 推荐内容是否过于重复？
- 高风险场景是否被正确拦截？
- AI Rerank 是否真的提升了结果？

3.2 需要新增的数据记录

建议在每次推荐决策时保存完整 trace：

```
RecommendationTrace = {  
  "request_id": "...",  
  "conversation_id": "...",  
  "turn_id": 12,  
  
  "user_input": "...",  
  "emotion_state": {...},  
  "risk_state": {...},  
  "user_profile": {...},  
  "conversation_summary": {...},  
  
  "gate_inputs": {  
    "emotion_intensity": 0.72,  
    "risk_score": 0.4,  
    "intent_score": 0.75,  
    "trend_score": 0.35,  
    "preference_score": 0.45,  
    "cooldown_penalty": 0.18  
  },  
  "gate_output": {  
    "score": 0.51,  
    "should_recommend": true,  
    "recommend_type": "soft",  
    "reason_codes": [...]  
  },  
  
  "query_variants": [...],  
  "retrieved_candidates": [...],  
  "ranked_candidates": [...],  
  "rerank_output": [...],  
}
```

```
"final_recommendations": [...],

"safety_override": false,
"latency_ms": 860
}
```

这一步非常重要。没有 trace，就无法判断推荐问题出在门控、召回、排序、Rerank 还是安全过滤。

3.3 建立离线评测集

先构造 100 条左右的推荐评测样本。每条样本包含：

```
{
  "user_input": "我最近论文压力很大，晚上也睡不着",
  "emotion": "学业压力/焦虑/失眠",
  "risk_level": "LEVEL_1",
  "intent": "seeking_relief",
  "expected_gate": "soft",
  "expected_categories": ["academic", "stress_management", "sleep", "relaxation"],
  "forbidden_categories": [],
  "notes": "适合推荐短时放松或睡眠支持内容"
}
```

样本建议覆盖：

场景	数量
学业压力	15
求职焦虑	15
失眠	10
人际冲突	10
自我怀疑	10
轻度抑郁情绪	10
愤怒/烦躁	8
单纯闲聊，不应推荐	10
Level 2 高风险支持	8
Level 3 危机场景	4

3.4 评测指标

建议定义以下指标：

门控指标

指标	含义
Gate Accuracy	hard / soft / none / safety_only 是否判断正确
Recommend Precision	系统推荐的轮次中，有多少是真的应该推荐
Recommend Recall	应该推荐的轮次中，有多少被系统捕捉到
Hard Trigger Rate	hard 推荐比例，防止过度打扰
Safety Block Rate	高风险场景下是否正确拦截推荐

召回与排序指标

指标	含义
Recall@5	Top 5 是否包含合适内容
nDCG@5	推荐排序是否合理
MRR	第一个合适内容排在多前
Duplicate Rate	近期重复推荐比例
Category Coverage	是否覆盖足够多情绪和场景
Safety Violation Rate	是否推了不适合风险等级的内容

工程指标

指标	含义
Avg Latency	平均推荐耗时
Rerank Failure Rate	AI Rerank 失败率
Fallback Rate	兜底搜索触发率
Empty Result Rate	无结果比例

3.5 交付物

v4.4 最终应该产出：

```
backend/recommendation_trace.py
backend/recommendation_eval.py
data/recommend_eval_set.json
reports/recommend_eval_report.md
tests/test_m2_recommend_eval.py
```

4. 阶段二：v4.5 门控策略校准

4.1 目标

对 RecommendGate 的权重和阈值进行系统调参。

当前门控公式已经可配置，但 `hard_threshold=0.58`、`soft_threshold=0.22` 仍然主要是经验值。下一步要用离线评测集来验证这些阈值是否合理。

4.2 具体工作

任务 1：网格搜索阈值

对 `hard_threshold` 和 `soft_threshold` 做 grid search：

```
soft_thresholds = [0.15, 0.20, 0.22, 0.25, 0.30]
hard_thresholds = [0.50, 0.55, 0.58, 0.62, 0.68]
```

评估每组配置下：

- 推荐准确率
- hard 推荐比例
- none 误判率
- Level 2 / Level 3 安全稳定性

任务 2：拆分 hard 与 soft 判定逻辑

当前 hard 主要依赖总分和意图/情绪条件。建议进一步明确：

```
hard 推荐：
用户明确求助 + 情绪强度较高 + 非高风险 + 冷却期允许

soft 推荐：
用户表达压力/困扰，但没有明确要求资源，适合自然语言轻提示

none：
闲聊、信息问答、刚刚推荐过、用户明显拒绝推荐
```

任务 3：加入“用户拒绝推荐”信号

如果用户说过：

- “不用推荐”
- “别给我推东西”
- “我不想看文章”
- “这些对我没用”

应该写入用户画像中的 recommendation_preference:

```
recommendation_preference = {  
    "accepts_hard_recommendation": false,  
    "preferred_types": ["exercise", "audio"],  
    "rejected_types": ["article"],  
    "last_rejection_turn": 18  
}
```

然后在 RecommendGate 中加入拒绝惩罚。

4.3 交付物

```
scripts/tune_recommend_gate.py  
reports/recommend_gate_calibration_report.md  
backend/recommend_gate.py 更新  
tests/test_m2_recommend_gate_calibration.py
```

5. 阶段三：v4.6 内容库扩展与标签体系升级

5.1 目标

把内容库从“可用”扩展到“足够覆盖真实场景”。

当前 27 条内容适合测试和 demo，但如果要支撑真实对话，内容规模偏小。下一步建议扩展到 80–120 条。

5.2 内容库扩展方向

类别	当前问题	建议目标
学业压力	需要更细分	15–20 条
求职焦虑	应单独加强	15 条
失眠	需要短音频/练习	10–15 条
人际冲突	需要沟通练习	10–15 条
自我怀疑	需要认知重评内容	10 条
情绪低落	需要低负担内容	10 条
愤怒/烦躁	需要即时调节练习	8–10 条
决策迷茫	需要规划工具	8–10 条

5.3 标签体系升级

建议新增几类字段：

```
{
  "therapeutic_goal": [
    "relief",
    "reflection",
    "planning",
    "behavior_activation",
    "sleep_support",
    "communication"
  ],
  "user_effort": "low / medium / high",
  "time_to_effect": "immediate / short_term / long_term",
  "interaction_style": "read / listen / write / practice / plan",
  "suitable_stage": "early_conversation / repeated_issue / action_planning",
  "avoid_when": [
    "high_risk",
    "user_rejected_articles",
    "severe_insomnia"
  ]
}
```

这样排序层就不只根据情绪匹配，还能根据用户当下状态判断内容是否“可执行”。

5.4 内容质量分级

建议给每条内容增加人工质量分：

```
quality_score = {
  "clarity": 0.9,
  "evidence_level": 0.8,
  "actionability": 0.9,
  "emotional_tone": 0.85,
  "safety": 1.0
}
```

其中 safety 应作为硬约束，不建议只进入排序加权。

5.5 交付物

```
data/content_db_v2.json
scripts/validate_content_schema.py
scripts/content_coverage_report.py
```

```
reports/content_library_upgrade_report.md
tests/test_m2_content_schema_v2.py
```

6. 阶段四：v4.7 用户反馈闭环

6.1 目标

让推荐模块知道用户是否接受推荐。

推荐系统最重要的不是“我认为这个内容匹配”，而是“用户是否觉得有用”。因此需要建立反馈数据。

6.2 反馈类型

先不需要复杂行为数据，可以从轻量反馈开始：

```
RecommendationFeedback = {
    "recommendation_id": "...",
    "item_id": "...",
    "feedback_type": "click / save / dismiss / helpful / not_helpful",
    "explicit_rating": 1,
    "timestamp": "...",
    "conversation_context": {...}
}
```

6.3 前端交互建议

推荐卡片下方可以有：

有帮助 / 不适合我 / 以后少推荐这类 / 换一个

这些反馈分别影响：

用户反馈	系统动作
有帮助	提升该类型、标签、难度的 preference
不适合我	降低该内容和相似内容权重
以后少推荐这类	写入 rejected_types / rejected_categories
换一个	保持推荐意图，但触发重排并排除当前 item

6.4 画像更新逻辑

```
if feedback == "helpful":
    profile.preferred_content_types += item.type
    profile.preferred_categories += item.category
    profile.preferred_duration_range = update_duration(item.duration)

if feedback == "not_helpful":
    profile.rejected_content_ids += item.id
    profile.rejected_categories += item.category
```

不要一条反馈就大幅改变画像，建议使用滑动平均或计数阈值。

6.5 交付物

```
backend/recommendation_feedback.py
backend/profile_preference_updater.py
api/recommendation_feedback endpoint
tests/test_p0_recommendation_feedback.py
reports/recommendation_feedback_design.md
```

7. 阶段五：v5.0 轻量学习排序模型

7.1 目标

当有了一定推荐日志和反馈数据后，可以把排序层从纯规则加权升级为“规则 + 学习排序”。

不建议现在立刻做模型化排序，因为当前缺少真实反馈数据。更合理的顺序是：

```
先打日志 → 再收反馈 → 再训练排序模型
```

7.2 模型输入特征

可以直接复用现有 12 维子分：

```
features = [
    retrieval_score,
    emotion_match_score,
    keyword_match_score,
    concern_match_score,
    stage_match_score,
    risk_match_score,
    preference_score,
```

```
    actionability,  
    repetition_penalty,  
    dedup_penalty,  
    popularity_component,  
    priority  
]
```

再加一些上下文特征：

```
context_features = [  
    emotion_intensity,  
    risk_level,  
    intent_score,  
    conversation_turn,  
    recent_recommendation_count,  
    user_accept_rate,  
    item_accept_rate  
]
```

7.3 可选模型

第一版不建议上复杂深度模型，建议用：

```
Logistic Regression / LightGBM / XGBoost / LambdaMART
```

预测目标可以是：

```
P(user_helpful | user_state, item)
```

也就是用户认为推荐有帮助的概率。

7.4 排序融合方式

不要完全替换规则排序，而是做融合：

```
final_score =  
    0.65 * rule_score  
    + 0.35 * learned_score
```

这样系统仍然保留可解释性和安全控制。

7.5 交付物

```
models/recommend_ranker.pkl
scripts/train_recommend_ranker.py
scripts/evaluate_recommend_ranker.py
backend/learned_recommender.py
reports/recommend_ranker_v1_report.md
```

8. 推荐模块最终目标架构

下一阶段完成后，推荐模块应该变成：

```
用户输入
↓
情绪识别 / 风险评估 / 意图识别 / 用户画像
↓
RecommendGate
↓
推荐 trace 记录
↓
多路查询召回
↓
BM25 + 语义检索 + RRF
↓
规则排序
↓
AI Rerank
↓
安全过滤
↓
推荐卡片 / soft 推荐
↓
用户反馈
↓
画像更新 + 排序模型训练
```

也就是形成完整闭环：

```
推荐决策 → 推荐结果 → 用户反馈 → 偏好更新 → 排序优化
```

9. 优先级排序

最高优先级

1. 推荐 trace 日志
2. 离线评测集
3. 门控阈值调参
4. 内容库扩展到 80+
5. 用户反馈接口

暂时不要优先做

1. 不要马上上复杂深度推荐模型
 2. 不要继续堆大量 hard-coded 规则
 3. 不要在没有评测集的情况下继续调权重
 4. 不要让 AI Rerank 直接决定安全相关推荐
 5. 不要把 Level 2 / Level 3 场景交给普通推荐逻辑处理
-

10. 建议开发顺序

第一步：补日志

实现 `RecommendationTrace`，把每次推荐决策完整记录下来。

第二步：做评测集

构造 100 条标准推荐场景，覆盖 should recommend、should not recommend、soft、hard、safety_only、Level 3 bypass。

第三步：写评测脚本

用同一批样本反复跑不同配置，输出指标报告。

第四步：调门控

根据评测结果重新选择 hard_threshold、soft_threshold 和冷却惩罚。

第五步：扩内容库

把内容库扩展到 80-120 条，并补充 therapeutic_goal、user_effort、time_to_effect、interaction_style 等字段。

第六步：接反馈

前端增加“有帮助 / 不适合 / 换一个”反馈入口，后端写入推荐反馈表。

第七步：训练轻量排序模型

等积累足够反馈后，用现有子分和用户反馈训练 learning-to-rank 模型。

11. 版本里程碑

v4.4：推荐评测版

完成：

- RecommendationTrace
- 推荐离线评测集
- Gate / Ranking / Safety 指标
- 自动生成评测报告

判断标准：

可以明确回答：
推荐模块在哪些场景表现好？
哪些场景误推荐？
哪些场景漏推荐？
问题出在门控还是排序？

v4.5：门控校准版

完成：

- 阈值 grid search
- 冷却参数调优
- 用户拒绝推荐信号
- hard / soft 判定逻辑细化

判断标准：

推荐触发不再依赖经验阈值，而是有评测依据。

v4.6：内容库增强版

完成：

- 内容库扩展到 80–120 条
- 内容标签体系升级
- 内容覆盖度报告
- 内容 schema 校验

判断标准：

大部分常见压力/情绪场景都能召回多个合适候选。

v4.7: 反馈闭环版

完成：

- 推荐反馈 API
- 用户偏好更新
- rejected_types / preferred_types
- 推荐效果统计

判断标准：

系统能根据用户反馈减少无效推荐，增强偏好内容。

v5.0: 学习排序版

完成：

- 反馈数据训练集
- 轻量排序模型
- 规则分 + 模型分融合
- A/B 对比报告

判断标准：

推荐系统从静态规则排序升级为可学习排序。

12. 总结

推荐模块下一步最关键的不是“继续改算法”，而是建立推荐闭环。

当前 v4.3 已经完成了门控、召回、排序、安全过滤和测试覆盖，下一阶段应该按照以下路线推进：

可观测 → 可评测 → 可调参 → 可反馈 → 可学习

推荐模块最终要解决的问题是：

在合适的时机，以合适的方式，把合适的心理支持资源推荐给合适的用户，并且能根据用户反馈持续变好。

这也是它从普通 RAG 工具升级为真正个性化 Agent 推荐系统的关键。